A comprehensive guide and template for monotonic stack based problems

02 March 2025 09:32

⇒ Monotonic Stacks are generally used to solve questions of type
- next greater element
- next smaller element
- previous greater element
- previous smaller element

⇒ To keep things simple, while traversing through an array, will always go from left to right.

∘ What is monotonic stack?

⇒ There could be four types of monotonic stacks.

1) Strictly increasing: every element of stack is strictly greater than the previous element, ex [1, 4, 5, 8, 9]
2) Non-decreasing: every element of stack is greater than or equal to than the previous element. [1, 4, 5, 5, 8, 9, 9]
3) Strictly decreasing: every element of stack is strictly decreasing than the previous element. [9, 8, 5, 4, 1]
4) Non-increasing: every element of stack is lesser than or equal to the previous element. [9, 9, 8, 5, 5, 4, 1]

⇒ Will assume right most element is top of the stack & left most is bottom of stack.

∘ A generic template:

⇒ We can use the following template to keep the monotonic property alive through execution of program.

```
function buildMonoStack(arr) {
  // initialize an empty stack
  stack = [];

  // iterate through all the elements in the array
  for (i = 0 to arr.length - 1) {
    while (stack is not empty && element represented by stack top 'OPERATOR' arr[i]) {
      // if the previous condition is satisfied, we pop the top element
      let stackTop = stack.pop();

      // do something with stackTop here e.g.
      // nextGreater[stackTop] = i
    }

    if (stack.length) {
      // if stack has some elements left
      // do something with stack top here e.g.
      // previousGreater[i] = stack.at(-1)
    }

    // at the end, we push the current index into the stack
    stack.push(i);
  }

  // At all points in time, the stack maintains its monotonic property
}
```

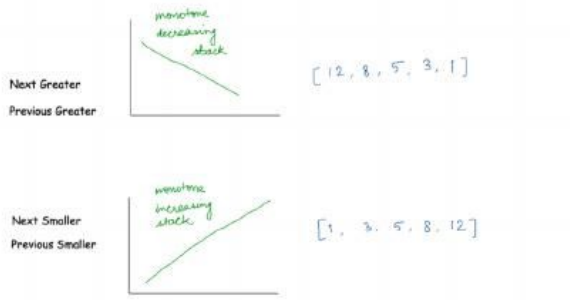
Notes about the template above

- We initialize an empty stack at the beginning.
- **The stack contains the index of items in the array, not the items themselves**
- There is an outer for loop and inner while loop.
- At the beginning of the program, the stack is empty, so we don't enter the while loop at first.
- The earliest we can enter the while loop body is during the second iteration of for loop. That's when there is at least an item in the stack.
- **At the end of the while loop, the index of the current element is pushed into the stack**
- The OPERATOR inside the while loop condition decides what type of monotonic stack are we creating.

- The OPERATOR could be any of the four - >, >=, <, <=

Time complexity - It can be argued that no element is accessed more than four times (a constant) - one, when comparing its value with the item in the stack (while conditional). two, when pushing the item in the stack. three, when comparing this item in the stack with the current item being iterated (while conditional again). four, when popping the item out of stack. As a result, *the time complexity of this algorithm is linear.* - O(n) where n is the number of elements in the array.

Space complexity - Because we are using an external data structure - stack. In the worst can it can be filled with all the elements in the array. *The space complexity is also linear.* - O(n) where n is the number of elements in the array.



⇒ In our implementation, finding NGr & PGr (next greater, previous greater) elements require building a monotone decreasing stack.

⇒ For finding next smaller & previous smaller requires building a monotone increasing stack.

⇒ To help remember this, think of this as an inverse relation - greater requires decreasing, smaller requires increasing.

1) Next Greater :

arr = [13, 8, 1, 5, 2, 5, 9, 7, 6, 12]

NGE (what is the just next greater element for an index).

next Greater Elements = [null, 9, 5, 9, 5, 9, 12, 12, 12, null]

No other element greater than 13 exists on right

9 is the next greater element for 8.

```
function findNextGreaterIndexes(arr) {
  // initialize an empty stack
  let stack = [];

  // initialize nextGreater array, this array hold the output
  // initialize all the elements are -1 (invalid value)
  let nextGreater = new Array(arr.length).fill(-1);

  // iterate through all the elements of the array
  for (let i = 0; i < arr.length; i++) {

    // while loop runs until the stack is not empty AND
    // the element represented by stack top is STRICTLY SMALLER than the current element
    // This means, the stack will always be monotonic non increasing (type 4)
    while (stack.length && arr[stack.at(-1)] < arr[i]) {

      // pop out the top of the stack, it represents the index of the item
      let stackTop = stack.pop();

      // as given in the condition of the while loop above,
      // nextGreater element of stackTop is the element at index i
      nextGreater[stackTop] = i;
    }

    // push the current element
    stack.push(i);
  }
  return nextGreater;
}
```

- Trace

arr = [13, 8, 1, 5, 2, 5, 9, 7, 6, 12]

To keep the stack decreasing, we need to keep popping all the elements which violate the monotonic non increasing property. decreasing + =

i.e elements, if a b then a < b

inner while (keep popping condition)

while stack and stack[-1] < arr[i]:

stackTop = stack.pop()

nextGreater[stackTop] = i

The incoming element is greater than current element, hence makes sense to mark the incoming element as the next greater element.

13? 8 ⇒ push 1 < 5 → keep popping all smaller elements

arr = [13, 8, 1, 5, 2, 5, 9, 7, 6, 12]

stack empty ⇒ push 8 > 1 ⇒ push

stack = [13, 8, 1] ← 5

1 —NGE→ 5

Similarly for others

stack = [13, 8, 2] ← 5 2 —NGE→ 5

stack = [13, 8] ← 9 8 —NGE→ 9

stack = [13, 9, 7, 6] ← 12

6 → 12

7 → 12

9 → 12

stack = [13, 12]

Unprocessed element defaults to None ⇒ No NGE for there.

2 Previous Greater

arr = [13, 8, 1, 5, 2, 5, 9, 7, 6, 12]

PGE = [-1, 13, 8, 8, 5, 8, 13, 9, 7, 13]

```
function findPreviousGreaterIndexes(arr) {
  // initialize an empty stack
  let stack = [];

  // initialize previousGreater array, this array hold the output
  // initialize all the elements are -1 (invalid value)
  let previousGreater = new Array(arr.length).fill(-1);

  // iterate through all the elements of the array
  for (let i = 0; i < arr.length; i++) {

    // while loop runs until the stack is not empty AND
    // the element represented by stack top is SMALLER OR EQUAL to the current element
    // This means, the stack will always be strictly decreasing (type 3) - because elements are popped when they are equal
    // so equal elements will never stay in the stack (definition of strictly decreasing stack)
    while (stack.length && arr[stack.at(-1)] <= arr[i]) {

      // pop out the top of the stack, it represents the index of the item
      let stackTop = stack.pop();
    }

    // after the while loop, only the elements which are greater than the current element are left in stack
    // this means we can confidentally decide the previous greater element of the current element i, that's stack top
    if (stack.length) {
      previousGreater[i] = stack.at(-1);
    }

    // push the current element
    stack.push(i);
  }
  return previousGreater;
}
```

- Trace

⇒ Same as above

Next when appending

if stack: (atleast one element is present)
Previous Greater [incoming] ← stack.top()

[3, 2] ← 1 ⇒ 1 —PGE→ 2
[3, 2, 1]

3 NGE & PGE at the same time:

```
function findNextAndPreviousGreaterIndexes(arr) {
  // initialize an empty stack
  let stack = [];

  // initialize previousGreater and nextGreater arrays
  let previousGreater = new Array(arr.length).fill(-1);
  let nextGreater = new Array(arr.length).fill(-1);

  // iterate through all the elements of the array
  for (let i = 0; i < arr.length; i++) {

    // while loop runs until the stack is not empty AND
    // the element represented by stack top is SMALLER OR EQUAL to the current element
    // This means, the stack will always be strictly decreasing (type 3) - because elements are popped when they are equal
    // so equal elements will never stay in the stack (definition of strictly decreasing stack)
    while (stack.length && arr[stack.at(-1)] <= arr[i]) {

      // pop out the top of the stack, it represents the index of the item
      let stackTop = stack.pop();

      // This is the only additional line added to the last approach
      // decides the next greater element for the index popped out from stack
      nextGreater[stackTop] = i;
    }

    // after the while loop, only the elements which are greater than the current element are left in stack
    // this means we can confidentally decide the previous greater element of the current element i, that's stack top
    if (stack.length) {
      previousGreater[i] = stack.at(-1);
    }

    // push the current element
    stack.push(i);
  }
  return [previousGreater, nextGreater];
}
```

⇒ Similar setting for NSE, PSE, etc.

□ Summary:

Summary

Let's summarize all the approaches here, to cement our learning.

Problem	Stack Type	Operator in while loop	Assignment Position
next greater	decreasing (equal allowed)	stackTop < current	inside while loop
previous greater	decreasing (strict)	stackTop <= current	outside while loop
next smaller	increasing (equal allowed)	stackTop > current	inside while loop
previous smaller	increasing (strict)	stackTop >= current	outside while loop

THE NGE AND PGE (or the NSE and PSE) need to be handled with care together as is shown for the edge case [2,2] it will fail : refer the discussion of (https://leetcode.com/discuss/study-guide/2347639/a-comprehensive-guide-and-template-for-monotonic-stack-based-problems) (or simply trace it), a simple if statement (inside the while loop) in the nge block should suffice to avoid counting duplicate eleemnts inside the while loop

Question) Why < in nge and <= in pge. => Basically we pop when < elements are considered and it avoids cases like [2,2] where 2's nge should be –1. And since after popping we do the pge. We should have the <= to ensure that the same element is popped.

Below are minimal examples (using the array [2, 2]) that illustrate how the **wrong** comparison (> vs. >=) breaks **previous greater** (PGE) and **next greater** (NGE) algorithms.

1. **Previous Greater** Fails with Only ">" (Should Use ">=")

Consider computing **Previous Greater Element** (PGE) for the array:

arr = [2, 2]

We want the **PGE** of arr[1] (the second 2) to be "no valid previous greater" (often marked -1).

Correct Approach (Uses <=)

Typical stack-based PGE logic (left to right) does:

1. **Pop** from the stack while arr[stack.top()] <= arr[i].
2. If the stack is not empty, arr[stack.top()] is the PGE.
3. Push i.

Let's walk through with <=:

- **i = 0**
  - Stack is empty. Push index 0.
  - Stack: [0] (top → 0)
  - **i = 1** (current element = arr[1] = 2)
    - Check top of stack: arr[0] = 2.
    - 2 <= 2? Yes ⇒ **pop** index 0.
    - Stack becomes empty.
    - No top left ⇒ PGE for arr[1] = -1.
    - Push 1.
    - Stack: [1]

End result: **PGE of the second 2** is -1, which is correct.

Wrong Approach (Uses Only <)

Now suppose we incorrectly pop while arr[stack.top()] < arr[i]:

- **i = 0**
  - Stack empty, push 0.
  - Stack: [0]
  - **i = 1** (current element = arr[1] = 2)
    - Check top of stack: arr[0] = 2.
    - Is 2 < 2? No ⇒ **do not pop**.
    - We then say PGE for arr[1] is arr[stack.top()] = 2.
      - That is **incorrect**, because the previous element (2) is **not strictly greater** than the current 2.
    - Push 1.
    - Stack: [0, 1]

End result: We incorrectly get **PGE of the second 2** as 2, which is wrong for a "strictly greater" definition.

Hence, **for PGE**, we must pop elements that are **less or equal** (<=) to the current value—otherwise duplicates will slip through and be (incorrectly) labeled as "previous greater."

2. **Next Greater** Fails with ">=" (Should Use ">")

Now consider computing **Next Greater Element** (NGE) for the same array [2, 2].

We want the **NGE** of the first 2 to be "none" (often -1), because there is no strictly larger element to its right—only another 2.

Correct Approach (Uses <)

Typical stack-based NGE logic (left to right) does:

1. While arr[stack.top()] < arr[i], pop from the stack and assign arr[i] as their "next greater."
2. Push i.

Walkthrough with <:

- **i = 0**
  - Stack empty, push 0.
  - Stack: [0]
  - **i = 1** (current = 2)
    - Check arr[stack.top()] = arr[0] = 2.
    - Is 2 < 2? No ⇒ do not pop.
    - Push 1.
    - Stack: [0, 1]

At the end, any indices left in the stack have no next greater ⇒ **NGE** of arr[0] is -1, and **NGE** of arr[1] is also -1. That's correct for "strictly greater."

Wrong Approach (Uses <=)

Now if we incorrectly pop while arr[stack.top()] <= arr[i]:

- **i = 0**
  - Stack empty, push 0.
  - Stack: [0]
  - **i = 1** (current = 2)
    - arr[stack.top()] = arr[0] = 2.
    - Is 2 <= 2? Yes ⇒ **pop** index 0.
      - While popping, we assign "next greater" of arr[0] to arr[1] ⇒ incorrectly record NGE(2) = 2.

- Push 1.
- Stack: [1]

End result: We incorrectly claim that the **next greater** for the first 2 is the second 2—even though they are equal, not greater.

Hence, **for NGE**, we must pop elements that are **strictly less** (<) than the current value—otherwise equal elements will also get popped, incorrectly marking them as if they'd found a larger value.

Summary

- **PGE** (previous greater) must remove (pop) elements that are **less or equal** (<=) to ensure the top is truly a *strictly* greater element.
- **NGE** (next greater) must remove elements that are **strictly less** (<) so that duplicates do not falsely claim each other as "next greater."

The minimal example [2, 2] makes both errors very obvious:

- Using only ">" for PGE allows equal duplicates to remain, so the PGE is wrong.
- Using ">=" for NGE pops duplicates, so the NGE is wrong.

These subtleties ensure that "strictly greater" means strictly, not equal.